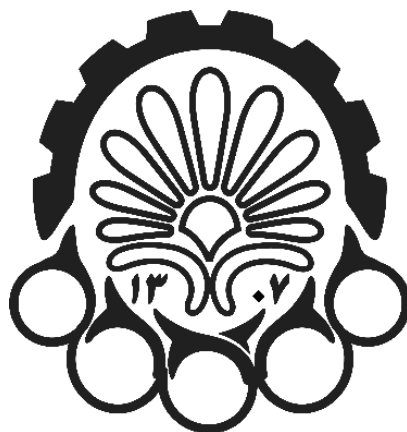




دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

دانشکده مهندسی کامپیوتر

پروژه پایانی درس آزمون نرم افزار

طراح پروژه:

محمد حمیدی مقدم

ارتباط با طراح پروژه:

@Mohammad0_0hm

استاد درس:

دکتر فائزه گوهری

بهار ۱۴۰۵

فهرست مطالب

۱. مقدمه	۳
۲. بخش آزمون جعبه سفید (White-Box Testing)	۴
۱.۲ رسم گراف کنترل جریان	۴
۲.۲ استخراج مسیرهای گراف کنترل جریان	۵
۳.۲ طراحی تست کیس بر اساس معیار All-DU-Paths Coverage	۵
۴.۲ پیاده سازی Unit Test ها	۵
۵.۲ محاسبه پوشش تست ها	۶
۳. بخش آزمون جعبه سیاه (Black-Box Testing)	۷
۱.۳ مبنای طراحی تست در بخش جعبه سیاه	۷
۲.۳ تکنیک های پیشنهادی طراحی تست در جعبه سیاه	۸
۳.۳ طراحی تست کیس ها	۹
۴.۳ اجرای تست ها و ثبت نتایج	۹
۴. بخش امتیازی پروژه	۱۰
۱.۴ تست Mutation	۱۰
۲.۴ تست Load و Performance	۱۱

۱. مقدمه

هدف کلی این پروژه، آشنایی عملی با فرآیندهای آزمون نرم افزار از طریق بررسی و ارزیابی یک سامانه مدیریت بودجه و هزینه ها می باشد. در این پروژه تلاش می شود دانشجویان با اجرای انواع مختلف آزمون ها در مراحل گوناگون توسعه نرم افزار، دید جامعی نسبت به نقش تست در تضمین کیفیت نرم افزار به دست آورند.

با توجه به اینکه آزمون های نرم افزار را می توان از یک دیدگاه کلی به دو دسته Black Box و White Box تقسیم کرد، در این پروژه تلاش می شود هر دو نوع آزمون مورد استفاده قرار گیرند. در آزمون های Black Box، عملکرد سامانه بدون توجه به جزئیات پیاده سازی و صرفاً بر اساس ورودی ها، خروجی ها و نیازمندی های سیستم بررسی می شود. در مقابل در آزمون های White Box، ساختار داخلی کد، مسیرهای اجرایی، شرط ها، متدها و منطق پیاده سازی شده مورد تحلیل قرار می گیرد.

سامانه مورد استفاده در این پروژه، یک سیستم مدیریت بودجه و هزینه ها است که با استفاده از Java 17، فریم ورک Spring Boot و Gradle توسعه داده شده است. این سامانه امکاناتی مانند مدیریت کاربران، ثبت دوستان، تعریف دسته بندی هزینه ها، ثبت هزینه ها، تقسیم هزینه بین افراد مشارکت کننده و مشاهده اطلاعات مربوط به هزینه ها و بدهی ها را فراهم می کند.

برای آشنایی اولیه با برنامه، ابتدا باید فایل docker compose که در پروژه قرار گرفته را اجرا کنید. با انجام این کار، یک نسخه از postgres با متغیرهای محیطی لازم به همراه یک volume برای ذخیره سازی داده های دیتابیس ایجاد می شود. سپس با اجرای دستور `gradlew bootRun`، برنامه اجرا شده، جداول مورد نیاز را به وسیله flyway ایجاد می کند و آماده دریافت دستورات از طریق پورت ۸۰۸۰ می شود. همچنین swagger پروژه نیز در آدرس زیر قابل مشاهده است:

`localhost:8080/swagger-ui.html`

۲. بخش آزمون جعبه سفید (White-Box Testing)

در این بخش از پروژه، هدف آشنایی عملی دانشجویان با آزمون جعبه سفید و تحلیل ساختار داخلی برنامه در سطحی پیشرفته‌تر می‌باشد. در آزمون جعبه سفید، طراحی تست‌ها صرفاً بر اساس ورودی و خروجی سیستم انجام نمی‌شود، بلکه ساختار داخلی کد، مسیرهای اجرایی، فراخوانی بین متدها، وابستگی بین لایه‌ها، تعریف و استفاده متغیرها، شرط‌ها، حلقه‌ها و مدیریت خطاها نیز مورد بررسی قرار می‌گیرد.

در این پروژه، تمرکز بخش جعبه سفید بر تحلیل متدهای موجود در Controllerهای سامانه و مسیرهای اجرایی آنها در لایه‌های پایین‌تر مانند Service است. شما باید برای هر متد موجود در Controllerها، یک Interprocedural Control Flow Graph یا ICFG رسم کنید. این گراف باید نشان دهد که اجرای هر درخواست از نقطه ورود در Controller آغاز شده و سپس از طریق فراخوانی متدهای Service، ValidationService، Repository و سایر متدهای مرتبط چگونه در برنامه جریان پیدا می‌کند.

پس از رسم ICFG، باید تمام DU Pairهای ممکن را استخراج کنید. منظور از DU Pair، ارتباط بین محل تعریف یک متغیر (Definition) و محل استفاده آن (Use) است. در این پروژه، استفاده از معیار All-DU-Paths Coverage الزامی است؛ یعنی برای هر DU Pair استخراج‌شده، باید تمام مسیرهای بدون تعریف مجدد (Definition-Clear Paths) بین تعریف و استفاده آن متغیر شناسایی شده و تا حد امکان با تست مناسب پوشش داده شوند.

توجه: انجام تمامی مراحل این بخش برای Controllerهای ExpenseController و ExpenseParticipantController الزامی است. نمره امتیازی این بخش نیز منوط به انجام همین مراحل برای سه Controller دیگر است.

۱.۲ رسم گراف کنترل جریان

در این بخش، شما باید تمام متدهای موجود در Controllerهای پروژه را به عنوان نقطه شروع تحلیل در نظر بگیرید. هر متد Controller یک نقطه ورود به سیستم محسوب می‌شود زیرا درخواست‌های کاربر یا کلاينت از طریق این متدها وارد برنامه شده و سپس به لایه‌های دیگر منتقل می‌شوند.

پس از شناسایی متدهای Controller، باید برای هر متد یک Interprocedural Control Flow Graph یا ICFG رسم کنید. ICFG نوعی گراف کنترل جریان است که علاوه بر جریان کنترل داخل یک متد، فراخوانی بین متدهای مختلف را نیز نمایش می‌دهد. در این گراف، مسیر اجرای برنامه از متد Controller آغاز شده و سپس وارد متدهای دیگر مانند Serviceها، Validatorها و Repositoryها می‌شود. در ICFG طراحی شده باید موارد زیر را در نظر بگیرید:

- Call Edges: با شروع هر فراخوانی، مسیر رسیدن به متد فراخوانده‌شده باید مشخص گردد.
- Return Edges: پس از پایان هر فراخوانی، مسیر بازگشت به متد فراخواننده باید مشخص گردد.
- Repository Calls: از آنجایی که عملیات‌های CRUD توسط Spring انجام می‌شود، فراخوانی‌های دیتابیزی را به صورت Black Box در نظر بگیرید (نیازی به طراحی Call Edge و Return Edge برای این موارد نیست).

برای رسم گراف می‌توان از ابزارهایی مانند Mermaid، Draw.io یا هر ابزار گرافیکی دیگر استفاده کرد. مهم این است که گراف رسم‌شده با ساختار واقعی کد مطابقت داشته باشد. برای این بخش در مستند پروژه، برای هر متد Controller باید ICFG مربوط به آن قرار داده شود.

۲.۲ استخراج مسیرهای گراف کنترل جریان

پس از رسم ICFG، باید جریان داده را در مسیرهای اجرایی تحلیل کنید. در این مرحله باید مشخص شود هر متغیر در کدام نقطه تعریف و در کدام نقطه استفاده می‌شود. در مستند پروژه، باید برای هر متد Controller و مسیرهای مرتبط با آن، جدول Definition و Use تهیه کنید. همچنین پس از استخراج جدول فوق باید تمام DU Pair های ممکن را استخراج کنید. برای هر DU Pair موارد زیر باید مشخص شود:

- متغیر مورد نظر
- محل Definition
- محل Use
- مسیر یا مسیرهای Definition-Clear بین Definition و Use

۳.۲ طراحی تست کیس بر اساس معیار All-DU-Paths Coverage

معیار اصلی پوشش در این بخش، All-DU-Paths Coverage است. بر اساس این معیار، برای هر DU Pair باید تمام مسیرهای Definition-Clear از محل تعریف متغیر تا محل استفاده آن متغیر پوشش داده شوند. منظور از Definition-Clear Path مسیری است که در آن، بین Definition و Use مورد نظر، تعریف مجددی از همان متغیر وجود نداشته باشد.

برای این بخش باید برای هر DU Pair استخراج شده، مسیرهای قابل اجرای Definition-Clear را شناسایی کرده و تست کیس‌هایی طراحی کنید که این مسیرها را اجرا کنند. بنابراین ممکن است برای یک DU Pair، چند تست کیس لازم باشد تا تمام مسیرهای مربوط به آن پوشش داده شوند. هر Test Case باید شامل موارد زیر باشد:

- نام Controller و متد مربوطه
- DU Pair های پوشش داده شده
- DU Path های پوشش داده شده
- پیش شرط‌ها

۴.۲ پیاده‌سازی Unit Test ها

برای بررسی مسیرهای بین‌رویه‌ای و ارتباط واقعی بین لایه‌ها، از آنجایی که پروژه از PostgreSQL استفاده می‌کند، پیاده‌سازی Test Container برای اجرای تست‌ها الزامی است. Test Container امکان اجرای یک نمونه واقعی PostgreSQL داخل Docker را هنگام اجرای تست‌ها فراهم می‌کند.

پس از پیاده‌سازی Test Container، تست‌های واحد باید برای پوشش DU Path های استخراج شده پیاده‌سازی شوند. برای انجام این کار باید به واسطه کتابخانه‌های موجود، درخواست به صورت HTTP ایجاد و به متد مدنظر از Controller ارسال بشود. انتظار می‌رود مسیرهای موفق و مسیرهای خطا به صورت کامل تست بشوند. همچنین معماری تمیز کلاس‌های تست (استفاده صحیح از متدهای Before و After، آزادسازی منابع در انتهای تست و ...) برای دریافت نمره کامل این بخش الزامی است.

۵.۲ محاسبه پوشش تست‌ها

پس از پیاده‌سازی تست‌ها، باید میزان پوشش تست‌های خود را محاسبه کنید. برای این کار می‌توان از ابزار JaCoCo استفاده کرد. گزارش JaCoCo می‌تواند موارد زیر را نشان دهد:

- Line Coverage
- Branch Coverage
- Method Coverage
- Class Coverage
- Instruction Coverage

با این حال، باید توجه شود که JaCoCo به صورت مستقیم معیار All-DU-Paths Coverage را گزارش نمی‌کند. بنابراین باید علاوه بر گزارش JaCoCo، یک جدول تحلیلی ارائه دهید که نشان دهد هر DU Pair و هر DU Path توسط کدام تست‌کیس پوشش داده شده است. یعنی گزارش نهایی پوشش باید شامل دو بخش باشد:

- پوشش کد با JaCoCo
- پوشش DU Path ها به صورت دستی و تحلیلی

۳. بخش آزمون جعبه سیاه (Black-Box Testing)

در این بخش از پروژه، هدف آشنایی عملی دانشجویان با آزمون جعبه سیاه و طراحی تست بدون اتکا به ساختار داخلی کد می‌باشد. در آزمون جعبه سیاه، تست‌کننده به جزئیات پیاده‌سازی، کلاس‌ها، متدها، شرط‌ها و مسیرهای داخلی برنامه توجهی ندارد و تنها بر اساس نیازمندی‌ها، ورودی‌ها، خروجی‌های مورد انتظار، قوانین کسب‌وکار، قرارداد API ها و رفتار قابل مشاهده سیستم، تست‌ها را طراحی و اجرا می‌کند.

در این پروژه، بخش جعبه سیاه بر بررسی رفتار خارجی سامانه مدیریت بودجه و هزینه‌ها متمرکز است. شما باید سیستم را از دید کاربر، مصرف‌کننده API یا تحلیل‌گر نیازمندی‌ها بررسی کنید و برای تمامی قابلیت‌های اصلی پروژه مانند مدیریت کاربر، مدیریت دوستان، مدیریت دسته‌بندی‌ها، ثبت هزینه، تقسیم هزینه بین مشارکت‌کنندگان، بازیابی اطلاعات و مدیریت خطاها، تست‌کیس طراحی نمایید. در این بخش، معیار اصلی ارزیابی، صحت عملکرد سیستم در مواجهه با ورودی‌های معتبر و نامعتبر، تطابق خروجی با نیازمندی‌ها و کامل بودن سناریوهای تست است نه آگاهی از نحوه پیاده‌سازی داخلی.

بنابراین، در بخش جعبه سیاه باید نشان دهید که بدون بررسی کد منبع و تنها بر اساس مستندات، قرارداد API ها، نیازمندی‌های عملکردی و خروجی‌های قابل مشاهده، قادر به طراحی تست‌های دقیق، ساخت‌یافته و پوشش‌دهنده رفتارهای اصلی سیستم هستید. برای انجام این کار در مقدمه پروژه آدرس swagger برای شما قرار داده شده است.

۱.۳ مبنای طراحی تست در بخش جعبه سیاه

در این پروژه، دامنه آزمون جعبه سیاه شامل تمام قابلیت‌هایی است که از طریق API های پروژه در دسترس قرار می‌گیرند. بنابراین، شما باید تمامی متدهای موجود در Controller ها را به عنوان قابلیت‌های قابل مشاهده سیستم در نظر بگیرید و برای آن‌ها تست طراحی کنید. در این بخش، تمرکز روی این است که اگر یک کاربر یا کلاینت فقط به API ها دسترسی داشته باشد، آیا می‌تواند با ورودی‌های مختلف رفتار سیستم را ارزیابی کند و از صحت عملکرد آن اطمینان یابد یا خیر.

طراحی تست‌های جعبه سیاه باید بر اساس مستندات API پروژه (Swagger)، ورودی‌ها و خروجی‌های تعریف‌شده برای هر Endpoint، شهود شما نسبت به قوانین کسب‌وکار سامانه، محدودیت‌ها و اعتبارسنجی‌های مورد انتظار، کدهای وضعیت HTTP و ساختار پاسخ‌های موفق و خطا و ... انجام شود. برای این بخش از پروژه شما باید تمام قابلیت‌های قابل مشاهده سیستم را از طریق Controller ها و مستندات API استخراج کرده باشید و قوانین رفتاری سیستم را بدون بررسی کد و تنها از طریق نیازمندی‌ها و رفتار API ها مشخص کرده باشید.

برای مثال وقتی از یک سامانه ثبت مخارج استفاده می‌کنیم، در API ثبت هزینه انتظار می‌رود کاربر معتبر باشد، دسته‌بندی وجود داشته باشد، مبلغ مثبت باشد، لیست مشارکت‌کنندگان خالی یا دارای مقدار معتبر باشد و ... شما نیز باید صرفاً بر اساس همین رفتار مورد انتظار تست طراحی کنید، نه بر اساس اینکه در لایه Service چه کدی نوشته شده است.

۲.۳ تکنیک‌های پیشنهادی طراحی تست در جعبه سیاه

برای طراحی تست‌های جعبه سیاه تکنیک‌های استاندارد زیادی وجود دارند که در این پروژه دو مورد از آن‌ها معرفی می‌شوند. در این بخش، شما باید از این تکنیک‌ها برای طراحی تست‌ها بهره ببرید و نحوه‌ی استفاده از آن‌ها را به صورت کامل توضیح بدهید.

توجه: انجام تمامی مراحل بخش جعبه سیاه با استفاده از تکنیک اول (Decision Table Testing) برای تمامی Endpointها الزامی است. نمره امتیازی این بخش نیز منوط به انجام همین مراحل برای تکنیک دوم (Equivalence Partitioning) است.

۱.۲.۳ جدول تصمیم (Decision Table Testing)

این تکنیک زمانی مناسب است که چند شرط مختلف با هم ترکیب می‌شوند و خروجی سیستم به ترکیب آن‌ها وابسته است. برای مثال، در ثبت هزینه ممکن است شرایط زیر وجود داشته باشد:

- کاربر معتبر است یا نه
- دسته‌بندی معتبر است یا نه
- مبلغ معتبر است یا نه
- مشارکت‌کنندگان معتبر هستند یا نه

خروجی بسته به این ترکیب‌ها تغییر می‌کند. پس شما باید برای هر Endpoint یک جدول تصمیم مانند جدول زیر تهیه کنید:

نتیجه مورد انتظار	کاربر معتبر	دسته‌بندی معتبر	مبلغ معتبر	مشارکت‌های معتبر
ثبت موفق هزینه	✓	✓	✓	✓
خطای کاربر نامعتبر	✗	✓	✓	✓
خطای دسته‌بندی نامعتبر	✓	✗	✓	✓
خطای مبلغ نامعتبر	✓	✓	✗	✓
خطای دوست نامعتبر	✓	✓	✓	✗

۲.۲.۳ تقسیم‌بندی هم‌ارزی (Equivalence Partitioning)

در این تکنیک، هر یک از ورودی‌های Endpoint به گروه‌هایی تقسیم می‌شوند که انتظار می‌رود تمام اعضای هر گروه رفتار مشابهی ایجاد کنند. سپس به جای تست تمام مقادیر، چند نماینده از هر گروه انتخاب می‌شود تا این رفتار مشابه ارزیابی شود. برای مثال، برای فیلد مبلغ هزینه:

- دسته معتبر: مبلغ بزرگ‌تر از صفر
- دسته نامعتبر: مبلغ صفر
- دسته نامعتبر: مبلغ منفی
- دسته نامعتبر: مقدار خالی یا غیر عددی

شما باید برای تمامی فیلدهای هر Endpoint از این تکنیک استفاده کنید و گروه‌های مناسب برای هر یک ارائه بدهید.

۳.۳ طراحی تست کیس‌ها

همانطور که گفته شد، در مراحل پیش شما باید تمام قابلیت‌های قابل مشاهده سیستم را از طریق Controllerها و مستندات API استخراج کرده باشید و قوانین رفتاری سیستم را بدون بررسی کد و تنها از طریق نیازمندی‌ها و رفتار APIها مشخص کرده باشید. حال پس از شناسایی قابلیت‌ها و قوانین، شما باید تست کیس‌های جعبه سیاه را با توجه به تکنیک‌های ارائه شده طراحی کنید. هر تست کیس باید شامل موارد زیر باشد:

- قابلیت مورد آزمون
- تکنیک طراحی تست
- پیش شرط‌ها
- داده‌های ورودی
- خروجی مورد انتظار
- خروجی واقعی
- وضعیت تست: Pass یا Fail

با توجه به توضیحات ارائه شده در بخش قبل، هنگام طراحی تست کیس‌ها باید هم به تست‌های مثبت (Positive Test Cases) و هم به تست‌های منفی (Negative Test Cases) توجه کافی را داشته باشید. تست‌های مثبت برای بررسی این هستند که سیستم در شرایط معتبر و مورد انتظار، رفتار درستی دارد. تست‌های منفی نیز بسیار مهم هستند زیرا بخش قابل توجهی از کیفیت سیستم به نحوه مدیریت خطاها وابسته است.

۴.۳ اجرای تست‌ها و ثبت نتایج

پس از طراحی تست کیس‌ها، شما باید تست‌ها را اجرا کنید و نتایج را مستندسازی نمایید. از آنجا که پروژه مبتنی بر REST API است، آزمون جعبه سیاه نیز باید به صورت API Testing انجام شود. برای این کار می‌توانید درخواست‌های خود را به صورت Curl اجرا کنید و یا اینکه از swagger پروژه استفاده کنید. در تست APIها باید مواردی که در طراحی تست کیس‌ها نوشتید را بررسی کنید. برای هر تست باید مشخص شود:

- نتیجه واقعی چه بوده است؟
- آیا با نتیجه مورد انتظار مطابقت دارد؟
- در صورت شکست علت چیست؟

به صورت کلی در پایان بخش آزمون جعبه سیاه، انتظار می‌رود بتوانید نشان دهید که بدون نیاز به دسترسی به جزئیات داخلی کد، تنها بر اساس نیازمندی‌ها و رفتار خارجی سیستم، تست‌های مؤثر و ساخت یافته طراحی کرده‌اید. این تست‌ها باید نشان دهند که سیستم در سناریوهای معتبر و نامعتبر چگونه رفتار می‌کند و آیا پاسخ‌های آن با انتظارات تعریف شده مطابقت دارد یا خیر.

۴. بخش امتیازی پروژه

در کنار بخش‌های امتیازی آزمون جعبه سفید و جعبه سیاه، این پروژه دارای یک بخش امتیازی دیگر نیز می‌باشد. هدف از این بخش، آشنایی شما با تکنیک‌ها و ابزارهای پیشرفته‌تر آزمون نرم‌افزار و ایجاد فرصت برای تحلیل عمیق‌تر کیفیت تست‌ها و کیفیت عملکردی سیستم است. در این بخش، دو فعالیت امتیازی تعریف شده است:

- Mutation Testing
- Performance and Load Testing

۱.۴ تست Mutation

Mutation Testing یا آزمون جهشی، یکی از روش‌های ارزیابی کیفیت تست‌ها است. در این روش، شما باید تغییرات کوچکی در کد برنامه ایجاد کنید. برای مثال یک شرط منطقی را تغییر دهید، یک عملگر ریاضی را عوض کنید، مقدار بازگشتی را تغییر دهید یا یک شرط را حذف کنید. به هر یک از این نسخه‌های تغییر یافته، یک Mutant گفته می‌شود.

سپس تست‌های موجود روی این نسخه‌های تغییر یافته اجرا می‌شوند. اگر تست‌ها بتوانند تغییر ایجاد شده را تشخیص دهند و شکست بخورند، گفته می‌شود آن Mutant کشته شده است. اما اگر تست‌ها همچنان Pass شوند، آن Mutant زنده مانده است. زنده ماندن Mutant معمولاً نشان می‌دهد که تست‌های موجود به اندازه کافی دقیق نیستند یا Assertion‌های کافی ندارند.

هدف این بخش آن است که متوجه شوید Coverage بالا، به تنهایی نشانه کیفیت بالای تست نیست. ممکن است یک تست بخشی از کد را اجرا کند، اما رفتار واقعی آن را به درستی بررسی نکند. Mutation Testing به شما کمک می‌کند کیفیت واقعی تست‌ها را بهتر ارزیابی کنید.

برای انجام این بخش باید در مسیر اجرایی هر یک از Endpoint‌های برنامه، یک Mutation ایجاد کنید. سپس با اجرای مجدد تست‌ها Mutant‌های کشته شده و زنده مانده را شناسایی کنید و برای Mutant‌های زنده مانده، دلیل زنده ماندن آن‌ها را تحلیل کنید. نهایتاً تمامی این موارد را به همراه ارائه اسکرین‌شات مستندسازی کنید.

۲.۴ تست Load و Performance

Load Testing و Performance Testing به بررسی رفتار سیستم از نظر سرعت، پایداری و توان پاسخ‌گویی در شرایط مختلف بار می‌پردازند. در تست‌های عملکردی معمولی بررسی می‌شود که سیستم پاسخ درست می‌دهد یا نه، اما در تست کارایی بررسی می‌شود که سیستم با چه سرعتی، با چه پایداری و تحت چه میزان بار پاسخ می‌دهد.

در این پروژه، چون سیستم به صورت REST API پیاده‌سازی شده است، می‌توانید API‌های اصلی سامانه مدیریت بودجه را تحت بارهای مختلف تست کنید و رفتار سیستم را از نظر زمان پاسخ، تعداد درخواست موفق، نرخ خطا و توان پردازش تحلیل نمایید. هدف این بخش آن است که با مفاهیم عملی Performance Testing مانند Response Time, Throughput, Concurrent Users, Error Rate و Percentile آشنا شوید و بتوانید یک گزارش ساده ولی معنادار از وضعیت کارایی سیستم ارائه کنید.

برای انجام این کار شما می‌توانید از هر ابزار دلخواهی استفاده کنید. پس از انتخاب ابزار، ابتدا باید آن را به صورت کوتاه معرفی کنید و سپس درباره علت انتخاب ابزار توضیحی خلاصه ارائه دهید.

در این بخش شما باید ۳ API مهم را برای تست انتخاب کنید. انتخاب API‌ها باید با دلیل همراه باشد. برای مثال، API دریافت لیست هزینه‌ها ممکن است با افزایش داده‌ها کند شود، یا API ثبت هزینه به دلیل عملیات نوشتن در دیتابیس اهمیت بیشتری داشته باشد. سپس برای هر API، پس از مهیا کردن پیش‌شرط‌های آن، باید سناریوهای بار زیر را طراحی و اجرا کنید:

- سناریوی اول (بار سبک): هدف این سناریو بررسی رفتار سیستم در حالت عادی است. برای مثال ۵ کاربر همزمان به مدت ۳۰ ثانیه تا ۱ دقیقه به اجرای درخواست می‌پردازند. در این سناریو انتظار می‌رود نرخ خطای صفر یا نزدیک به صفر و زمان پاسخ قابل قبول داشته باشیم.
- سناریوی دوم (بار متوسط): هدف این سناریو بررسی رفتار سیستم در شرایط پرتراфик‌تر است. برای مثال ۲۰ تا ۵۰ کاربر همزمان به مدت ۱ تا ۳ دقیقه به اجرای درخواست می‌پردازند. در این سناریو انتظار می‌رود افزایش کنترل‌شده زمان پاسخ و عدم افزایش شدید خطا را مشاهده کنیم.
- سناریوی سوم (بار زیاد): هدف این سناریو بررسی رفتار سیستم در شرایط بحرانی‌تر است. برای مثال ۲۰۰ تا ۲۵۰ کاربر همزمان به مدت ۱ تا ۳ دقیقه به اجرای درخواست می‌پردازند. در این سناریو انتظار می‌رود افزایش محسوس زمان پاسخ و یا دریافت خطا را مشاهده کنیم.
- سناریوی چهارم (داده زیاد): هدف این سناریو بررسی اثر حجم داده روی API‌های خواندنی است. بنابراین از بین API‌های انتخابی، صرفاً برای مواردی که داده‌های دیتابیس را واکنشی می‌کنند این مورد را بررسی کنید. نمونه‌ای از انجام این کار این است که به صورت پیوسته تعداد زیادی expense برای یک کاربر ایجاد کنید و پس از هر بار، API دریافت لیست هزینه‌ها را اجرا کنید و به مقایسه زمان پاسخ‌ها بپردازید. در این سناریو انتظار می‌رود افزایش زمان پاسخ را مشاهده کنیم.
- سناریوی پنجم (تست مرزی بار): هدف این سناریو یافتن نقطه‌ای است که سیستم شروع به افت عملکرد می‌کند. برای مثال به صورت تدریجی تعداد کاربران (درخواست‌ها) را افزایش دهید و بررسی کنید که از چه نقطه‌ای زمان پاسخ یا نرخ خطا افزایش جدی پیدا می‌کند.

در گزارش Performance Testing برای هر تست باید شاخص‌های زیر ثبت شوند:

- تعداد کل درخواست‌ها
- تعداد درخواست‌های موفق و ناموفق
- Error Rate
- میانگین زمان پاسخ
- حداقل و حداکثر زمان پاسخ
- p90 و p95 زمان پاسخ
- Throughput یا تعداد درخواست در ثانیه
- تعداد کاربران همزمان
- مدت اجرای تست

برای اینکه نتایج تست قابل اعتمادتر باشند، تست‌ها را روی محیط ثابت اجرا کنید. همچنین در زمان اجرای تست، برنامه‌های سنگین دیگر را تا حد امکان ببندید. در گزارش نهایی، نباید فقط خروجی ابزار را قرار دهید بلکه لازم است نتایج را به صورت خلاصه تحلیل کنید.